

Résumé Complet NoSQL

CAP, ACID, CRUD pour Cassandra, MongoDB, Neo4j et Redis

AmineGR03

1^{er} février 2026

Table des matières

Introduction

Ce document présente les concepts fondamentaux des bases de données NoSQL et les opérations CRUD pour les principales technologies : Cassandra, MongoDB, Neo4j et Redis.

Théorème CAP

Définition

Le **théorème CAP** (Consistency, Availability, Partition tolerance) énonce qu'un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes :

Définition

Consistency (Cohérence) : Tous les nœuds voient les mêmes données au même moment. Après une mise à jour, tous les nœuds retournent immédiatement la même valeur.

Availability (Disponibilité) : Le système reste opérationnel et répond aux requêtes même en cas de défaillance de certains nœuds.

Partition tolerance (Tolérance aux pannes) : Le système continue de fonctionner même si la communication entre certains nœuds est interrompue (partition réseau).

Choix CAP par Base de Données

Base de données	C	A	P
Cassandra	×	✓	✓
MongoDB	✓	×	✓
Neo4j	✓	×	✓
Redis	✓	✓	×

TABLE 1 – Choix CAP des bases de données NoSQL

Explications Détaillées

Cassandra : AP (Availability + Partition tolerance)

- **Architecture** : Système distribué avec réplication
- **Choix** : Privilégie la disponibilité et la tolérance aux pannes
- **Cohérence** : Cohérence éventuelle (Eventual Consistency)
- **Avantages** : Haute disponibilité, tolérance aux pannes réseau
- **Inconvénients** : Les lectures peuvent retourner des données obsolètes temporairement
- **Niveau de cohérence** : Configurable (ONE, QUORUM, ALL)

MongoDB : CP (Consistency + Partition tolerance)

- **Architecture** : Réplication avec primary/secondary
- **Choix** : Privilégie la cohérence et la tolérance aux pannes
- **Disponibilité** : En cas de partition, le primary peut être indisponible
- **Avantages** : Données toujours cohérentes, tolérance aux pannes
- **Inconvénients** : Peut être indisponible lors d'une partition réseau
- **Transactions** : Supporte les transactions multi-documents (depuis v4.0)

Neo4j : CP (Consistency + Partition tolerance)

- **Architecture** : Cluster avec leader/follower
- **Choix** : Privilégie la cohérence et la tolérance aux pannes
- **Disponibilité** : En cas de partition, le leader peut être indisponible
- **Avantages** : Données toujours cohérentes, relations garanties
- **Inconvénients** : Peut être indisponible lors d'une partition
- **Transactions** : Supporte les transactions ACID

Redis : CA (Consistency + Availability)

- **Architecture** : Principalement en mémoire, peut être distribué
- **Choix** : Privilégie la cohérence et la disponibilité
- **Partition** : Redis Cluster supporte la partition tolerance
- **Avantages** : Données cohérentes, très rapide, haute disponibilité
- **Inconvénients** : Limité par la mémoire, partition tolerance optionnelle
- **Mode cluster** : Redis Cluster offre partition tolerance avec cohérence éventuelle

Propriétés ACID

Définition

Les propriétés **ACID** garantissent la fiabilité des transactions dans une base de données :

Définition

Atomicity (Atomicité) : Une transaction est "tout ou rien". Soit toutes les opérations réussissent, soit aucune n'est appliquée.

Consistency (Cohérence) : Une transaction amène la base de données d'un état cohérent à un autre état cohérent. Les contraintes d'intégrité sont respectées.

Isolation (Isolation) : Les transactions concurrentes ne se voient pas mutuellement. Chaque transaction s'exécute comme si elle était la seule.

Durability (Durabilité) : Une fois une transaction validée (committed), ses modifications persistent même en cas de panne système.

Support ACID par Base de Données

Base de données	A	C	I	D
Cassandra	✓	Partiel	Partiel	✓
MongoDB	✓	✓	✓	✓
Neo4j	✓	✓	✓	✓
Redis	✓	✓	Partiel	Partiel

TABLE 2 – Support ACID des bases de données NoSQL

Explications Détaillées

Cassandra et ACID

- **Atomicity** : ✓ Supportée au niveau de la partition (BATCH)
- **Consistency** : **Partielle** - Cohérence éventuelle par défaut, configurable
- **Isolation** : **Partielle** - Isolation au niveau de la partition
- **Durability** : ✓ Écritures persistées sur disque
- **Transactions** : Pas de transactions multi-partitions traditionnelles
- **LWT (Lightweight Transactions)** : Supporte les opérations conditionnelles atomiques

MongoDB et ACID

- **Atomicity** : ✓ Supportée au niveau du document (single-document) et multi-documents (depuis v4.0)
- **Consistency** : ✓ Garantie par les transactions et les contraintes
- **Isolation** : ✓ Niveaux d'isolation configurables (read uncommitted, read committed, snapshot)
- **Durability** : ✓ Journaling (WAL) et réplication
- **Transactions** : Transactions multi-documents depuis MongoDB 4.0
- **Replica Set** : Garantit la cohérence entre réplicas

Neo4j et ACID

- **Atomicity** : ✓ Toutes les opérations dans une transaction sont atomiques
- **Consistency** : ✓ Contraintes et règles de graphe garanties
- **Isolation** : ✓ Isolation complète des transactions
- **Durability** : ✓ Transactions journalisées et persistées
- **Transactions** : Support complet des transactions ACID
- **Graph Integrity** : Garantit l'intégrité des relations et nœuds

Redis et ACID

- **Atomicity** : ✓ Toutes les commandes sont atomiques, MULTI/EXEC pour transactions
- **Consistency** : ✓ Garantie par l'exécution atomique des commandes
- **Isolation** : **Partielle** - Pas d'isolation entre transactions (pas de verrous)
- **Durability** : **Partielle** - Optionnelle (AOF, RDB), par défaut en mémoire
- **Transactions** : MULTI/EXEC pour exécution atomique (pas de rollback)
- **Persistence** : AOF (Append Only File) ou RDB (snapshots) pour durabilité

Opérations CRUD

Définition

Les opérations **CRUD** sont les quatre opérations de base pour manipuler des données :

Définition

Create (Créer) : Insérer de nouvelles données dans la base.

Read (Lire) : Récupérer des données existantes.

Update (Mettre à jour) : Modifier des données existantes.

Delete (Supprimer) : Supprimer des données de la base.

Cassandra - CRUD

Create (INSERT)

```

1 -- Insertion simple
2 INSERT INTO utilisateurs (id, nom, email, age)
3 VALUES (uuid(), 'Dupont', 'dupont@email.com', 30);
4
5 -- Insertion avec TTL (Time To Live)
6 INSERT INTO utilisateurs (id, nom, email)
7 VALUES (uuid(), 'Martin', 'martin@email.com')
8 USING TTL 3600;
9
10 -- Insertion conditionnelle
11 INSERT INTO utilisateurs (id, nom, email)
12 VALUES (a1b2c3d4-e5f6-7890-abcd-ef1234567890, 'Dupont', 'dupont@email.
    com')
13 IF NOT EXISTS;
```

Read (SELECT)

```

1 -- S lection de tous les champs
2 SELECT * FROM utilisateurs;
3
4 -- S lection de champs sp cifiques
5 SELECT id, nom, email FROM utilisateurs;
6
7 -- S lection avec WHERE
8 SELECT * FROM utilisateurs WHERE id = a1b2c3d4-e5f6-7890-abcd-
    ef1234567890;
9
10 -- S lection avec LIMIT
11 SELECT * FROM utilisateurs LIMIT 10;
12
13 -- S lection avec COUNT
14 SELECT COUNT(*) FROM utilisateurs;
15
16 -- S lection avec ALLOW FILTERING (attention: co teux)
17 SELECT * FROM utilisateurs WHERE ville = 'Paris' AND age > 25
18 ALLOW FILTERING;
```

Update (UPDATE)

```

1 -- Mise      jour simple
2 UPDATE utilisateurs
3 SET age = 31, ville = 'Lyon'
4 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
5
6 -- Mise      jour avec TTL
7 UPDATE utilisateurs
8 USING TTL 7200
9 SET email = 'nouveau@email.com'
10 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
11
12 -- Mise      jour conditionnelle
```

```

13 UPDATE utilisateurs
14 SET age = 32
15 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
16 IF age = 31;
17
18 -- Mise à jour avec IF EXISTS
19 UPDATE utilisateurs
20 SET age = 33
21 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
22 IF EXISTS;

```

Delete (DELETE)

```

1 -- Suppression d'un document
2 DELETE FROM utilisateurs
3 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
4
5 -- Suppression de champs spécifiques
6 DELETE email, age FROM utilisateurs
7 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
8
9 -- Suppression conditionnelle
10 DELETE FROM utilisateurs
11 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
12 IF age = 30;
13
14 -- TRUNCATE (vider une table)
15 TRUNCATE utilisateurs;

```

Tableau Récapitulatif CRUD Cassandra

Opération	Commande
Create	INSERT INTO ... VALUES ...
Read	SELECT ... FROM ... WHERE ...
Update	UPDATE ... SET ... WHERE ...
Delete	DELETE FROM ... WHERE ...

TABLE 3 – Opérations CRUD Cassandra

MongoDB - CRUD

Create (Insert)

```
1 // Insertion d'un document
2 db.collection.insertOne({
3     nom: "Dupont",
4     age: 30,
5     ville: "Paris"
6 })
7
8 // Insertion de plusieurs documents
9 db.collection.insertMany([
10     {nom: "Martin", age: 25, ville: "Lyon"},
11     {nom: "Bernard", age: 35, ville: "Marseille"}
12 ])
13
14 // Insertion avec insert() (d pr ci mais fonctionne)
15 db.collection.insert({
16     nom: "Durand",
17     age: 28,
18     ville: "Toulouse"
19 })
```

Read (Find)

```
1 // Lecture de tous les documents
2 db.collection.find()
3
4 // Lecture avec formatage
5 db.collection.find().pretty()
6
7 // Lecture d'un seul document
8 db.collection.findOne({age: 30})
9
10 // Lecture avec filtre
11 db.collection.find({ville: "Paris"})
12
13 // Projection (s lection de champs)
14 db.collection.find({}, {nom: 1, age: 1, _id: 0})
15
16 // Lecture avec op rateurs
17 db.collection.find({age: {$gt: 25}})
18 db.collection.find({ville: {$in: ["Paris", "Lyon"]}})
```

Update (Update)

```
1 // Mise jour d'un document
2 db.collection.updateOne(
3     {nom: "Dupont"},
4     {$set: {age: 31}}
5 )
6
7 // Mise jour de plusieurs documents
8 db.collection.updateMany(
```

```

9      {ville: "Paris"},
10      {$set: {pays: "France"}}
11  )
12
13  // Opérateurs de mise à jour
14  {$set: {age: 30}}           // Définir une valeur
15  {$inc: {age: 1}}           // Incrémenter
16  {$unset: {ville: ""}}      // Supprimer un champ
17  {$push: {hobbies: "lecture"}} // Ajouter un tableau
18  {$pull: {hobbies: "lecture"}} // Retirer d'un tableau
19
20  // Replace (remplacement complet)
21  db.collection.replaceOne(
22      {nom: "Dupont"},
23      {nom: "Dupont", age: 32, ville: "Lyon"}
24  )

```

Delete (Delete)

```

1  // Suppression d'un document
2  db.collection.deleteOne({nom: "Dupont"})
3
4  // Suppression de plusieurs documents
5  db.collection.deleteMany({ville: "Paris"})
6
7  // Suppression de tous les documents
8  db.collection.deleteMany({})

```

Tableau Récapitulatif CRUD MongoDB

Opération	Commande
Create	insertOne(), insertMany()
Read	find(), findOne()
Update	updateOne(), updateMany()
Delete	deleteOne(), deleteMany()

TABLE 4 – Opérations CRUD MongoDB

Neo4j - CRUD

Create (CREATE, MERGE)

```

1 -- Cr er un n ud
2 CREATE (p:Person {nom: 'Dupont', age: 30, ville: 'Paris'})
3 RETURN p;
4
5 -- Cr er plusieurs n uds
6 CREATE
7     (p1:Person {nom: 'Dupont', age: 30}),
8     (p2:Person {nom: 'Martin', age: 25}),
9     (p3:Person {nom: 'Bernard', age: 35})
10 RETURN p1, p2, p3;
11
12 -- Cr er une relation
13 CREATE (p1:Person {nom: 'Dupont'})-[:CONNAIT]->(p2:Person {nom: 'Martin',
14     })
15 RETURN p1, p2;
16
17 -- MERGE (cr er si n'existe pas, trouver si existe)
18 MERGE (p:Person {nom: 'Dupont'})
19 ON CREATE SET p.age = 30, p.ville = 'Paris'
20 ON MATCH SET p.age = 31
21 RETURN p;

```

Read (MATCH)

```

1 -- MATCH simple
2 MATCH (p:Person)
3 RETURN p;
4
5 -- MATCH avec filtre
6 MATCH (p:Person {nom: 'Dupont'})
7 RETURN p;
8
9 -- MATCH avec WHERE
10 MATCH (p:Person)
11 WHERE p.age > 25
12 RETURN p;
13
14 -- MATCH avec relations
15 MATCH (p1:Person)-[:CONNAIT]->(p2:Person)
16 RETURN p1, p2;
17
18 -- MATCH avec profondeur variable
19 MATCH (p:Person)-[:CONNAIT*1..3]->(autre:Person)
20 RETURN p, autre;
21
22 -- OPTIONAL MATCH (jointure externe)
23 MATCH (p:Person)
24 OPTIONAL MATCH (p)-[:CONNAIT]->(ami:Person)
25 RETURN p, ami;

```

Update (SET)

```

1 -- SET simple
2 MATCH (p:Person {nom: 'Dupont'})
3 SET p.age = 31
4 RETURN p;
5
6 -- SET multiple
7 MATCH (p:Person {nom: 'Dupont'})
8 SET p.age = 31, p.ville = 'Lyon'
9 RETURN p;
10
11 -- Ajouter un label
12 MATCH (p:Person {nom: 'Dupont'})
13 SET p:Client
14 RETURN p;
15
16 -- Mettre à jour une relation
17 MATCH (p1:Person {nom: 'Dupont'})-[r:CONNAIT]->(p2:Person {nom: 'Martin'})
18 SET r.depuis = 2021
19 RETURN r;

```

Delete (DELETE, DETACH DELETE, REMOVE)

```

1 -- DELETE n ud (n cessite de supprimer les relations d'abord)
2 MATCH (p:Person {nom: 'Dupont'})
3 DELETE p;
4
5 -- DETACH DELETE (supprime n ud et toutes ses relations)
6 MATCH (p:Person {nom: 'Dupont'})
7 DETACH DELETE p;
8
9 -- DELETE relation
10 MATCH (p1:Person {nom: 'Dupont'})-[r:CONNAIT]->(p2:Person {nom: 'Martin'})
11 DELETE r;
12
13 -- REMOVE (supprimer propri t ou label)
14 MATCH (p:Person {nom: 'Dupont'})
15 REMOVE p.ville
16 RETURN p;
17
18 MATCH (p:Person {nom: 'Dupont'})
19 REMOVE p:Client
20 RETURN p;

```

Tableau Récapitulatif CRUD Neo4j

Opération	Commande
Create	CREATE, MERGE
Read	MATCH ... RETURN
Update	MATCH ... SET
Delete	MATCH ... DELETE, DETACH DELETE
Remove	MATCH ... REMOVE

TABLE 5 – Opérations CRUD Neo4j

Redis - CRUD

Create (SET, HSET, LPUSH, RPUSH, SADD, ZADD)

```

1 -- String
2 SET cle "valeur"
3 SET nom "Dupont"
4 SET age 30
5
6 -- Hash
7 HSET utilisateur:1 nom "Dupont"
8 HSET utilisateur:1 age 30
9 HSET utilisateur:1 ville "Paris"
10
11 -- List
12 LPUSH liste "element1"
13 RPUSH liste "element2"
14
15 -- Set
16 SADD tags "important"
17 SADD tags "urgent" "prioritaire"
18
19 -- Sorted Set
20 ZADD scores 100 "joueur1"
21 ZADD scores 200 "joueur2" 150 "joueur3"

```

Read (GET, HGET, LRANGE, SMEMBERS, ZRANGE)

```

1 -- String
2 GET cle
3 GET nom
4
5 -- Hash
6 HGET utilisateur:1 nom
7 HGETALL utilisateur:1
8
9 -- List
10 LRANGE liste 0 -1
11 LINDEX liste 0
12
13 -- Set
14 SMEMBERS tags
15 SISMEMBER tags "important"
16
17 -- Sorted Set
18 ZRANGE scores 0 -1

```

```
19 ZRANGE scores 0 -1 WITHSCORES
20 ZREVRANGE scores 0 -1
21 ZSCORE scores "joueur1"
```

Update (SET, HSET, LSET, ZADD)

```
1 -- String (remplace la valeur)
2 SET nom "Martin"
3
4 -- Hash
5 HSET utilisateur:1 age 31
6
7 -- List
8 LSET liste 0 "nouvelle_valeur"
9
10 -- Set (ajouter)
11 SADD tags "nouveau_tag"
12
13 -- Sorted Set (modifier score)
14 ZADD scores 250 "joueur1"
15 ZINCRBY scores 10 "joueur1"
```

Delete (DEL, HDEL, LPOP, RPOP, SREM, ZREM)

```
1 -- String
2 DEL cle
3 DEL nom age
4
5 -- Hash
6 HDEL utilisateur:1 ville
7 HDEL utilisateur:1 nom age
8
9 -- List
10 LPOP liste
11 RPOP liste
12 LREM liste 2 "val"
13
14 -- Set
15 SREM tags "important"
16 SREM tags "urgent" "prioritaire"
17
18 -- Sorted Set
19 ZREM scores "joueur1"
20 ZREM scores "joueur2" "joueur3"
```

Tableau Récapitulatif CRUD Redis

Type	Create	Read	Update	Delete
String	SET	GET	SET	DEL
Hash	HSET	HGET, HGETALL	HSET	HDEL
List	LPUSH/RPUSH	LRANGE	LSET	LPOP/RPOP
Set	SADD	SMEMBERS	SADD	SREM
Sorted Set	ZADD	ZRANGE	ZADD	ZREM

TABLE 6 – Opérations CRUD Redis par type de données

Comparaison CRUD entre Bases de Données

Tableau Comparatif

Opération	Cassandra	MongoDB	Neo4j	Redis
Create	INSERT INTO VALUES	insertOne() insertMany()	CREATE MERGE	SET/HSET/ LPUSH/SADD/ZADD
Read	SELECT ... FROM ... WHERE	find() findOne()	MATCH ... RETURN	GET/HGET/ LRANGE/SMEMBERS
Update	UPDATE ... SET ... WHERE	updateOne() updateMany()	MATCH ... SET	SET/HSET/ LSET/ZADD
Delete	DELETE FROM WHERE	deleteOne() deleteMany()	MATCH ... DELETE	DEL/HDEL/ LPOP/SREM/ZREM

TABLE 7 – Comparaison des opérations CRUD

Caractéristiques Spécifiques

Cassandra

- Syntaxe SQL-like (CQL)
- Requêtes limitées par la structure de la clé primaire
- Support des opérations conditionnelles (IF EXISTS, IF NOT EXISTS)
- TTL (Time To Live) pour expiration automatique
- BATCH pour opérations atomiques

MongoDB

- Syntaxe JavaScript/JSON
- Documents flexibles (schéma dynamique)
- Opérateurs de mise à jour riches (\$set, \$inc, \$push, etc.)
- Projection pour sélectionner des champs
- Agrégation pipeline pour requêtes complexes

Neo4j

- Syntaxe Cypher (déclarative)
- Focus sur les relations entre nœuds
- MERGE pour créer ou mettre à jour
- DETACH DELETE pour supprimer nœuds et relations
- Requêtes de graphe (chemins, profondeur variable)

Redis

- Commandes simples et directes
- Structures de données variées (String, Hash, List, Set, Sorted Set)
- Opérations atomiques par commande
- Transactions avec MULTI/EXEC
- TTL pour expiration automatique

Résumé et Tableaux Récapitulatifs

Tableau CAP

Base de données	Choix CAP	Type	Caractéristique principale
Cassandra	AP	Wide-column	Haute disponibilité, répartition
MongoDB	CP	Document	Cohérence forte, réplication
Neo4j	CP	Graph	Cohérence, relations garanties
Redis	CA*	Key-value	Performance, cohérence

TABLE 8 – Classification CAP des bases NoSQL

*Redis Cluster offre partition tolerance

Tableau ACID

Base de données	Support ACID	Notes
Cassandra	Partiel	Atomicité par partition, cohérence éventuelle
MongoDB	Complet	Transactions multi-documents (v4.0+)
Neo4j	Complet	Transactions ACID complètes
Redis	Partiel	Atomicité par commande, durabilité optionnelle

TABLE 9 – Support ACID des bases NoSQL

Tableau CRUD - Vue d'Ensemble

Opération	Cassandra	MongoDB	Neo4j	Redis
Create	INSERT INTO ... VALUES	insertOne() insertMany()	CREATE MERGE	SET HSET/ZADD
Read	SELECT ... FROM ... WHERE	find() findOne()	MATCH ... RETURN	GET HGETALL
Update	UPDATE ... SET ... WHERE	updateOne() updateMany()	MATCH ... SET	SET HSET
Delete	DELETE FROM ... WHERE	deleteOne() deleteMany()	MATCH ... DELETE	DEL HDEL

TABLE 10: Tableau comparatif complet CRUD

Conseils et Bonnes Pratiques

Choix de Base de Données

- **Cassandra** : Données distribuées, haute disponibilité, écritures intensives
- **MongoDB** : Documents flexibles, requêtes complexes, évolutivité horizontale
- **Neo4j** : Relations complexes, graphes sociaux, recommandations
- **Redis** : Cache, sessions, queues, données temporaires, performance

Considérations CAP

- **AP** : Accepte la cohérence éventuelle pour la disponibilité
- **CP** : Priorise la cohérence, peut sacrifier la disponibilité
- **CA** : Fonctionne bien en réseau local, limité en distribution

Considérations ACID

- Les bases NoSQL offrent souvent un compromis entre ACID et performance
- MongoDB et Neo4j offrent un support ACID complet
- Cassandra et Redis offrent des garanties partielles mais optimisées
- Choisir selon les besoins de cohérence de l'application

Opérations CRUD

- **Cassandra** : Attention aux requêtes (limitées par la clé primaire)
- **MongoDB** : Utiliser les index pour optimiser les requêtes
- **Neo4j** : Exploiter les relations pour des requêtes efficaces
- **Redis** : Choisir la bonne structure de données pour chaque cas d'usage

Fin du document